

ENGENHARIA DA QUALIDADE E CONFIABILIDADE

Aprofundando nos Conceitos de Confiabilidade segundo Sommerville.

Professor: Me. Sergio Souza Novak
Faculdade de Engenharia de Software



- 1 Disponibilidade de serviços e desafios
- 2 Problemas possíveis
- 3 Ponderações sobre os Problemas nos Sistemas
- 4 Custo de Detecção vs. Erros Residuais
- 5 Diferenças entre Confiabilidade e Disponibilidade
- 6 Sistema como Mapeamento de Entradas/Saídas
- 7 Impacto Real da Indisponibilidade
- 8 Dependabilidade
- 9 Métricas de Confiabilidade
- 10 Métricas de Disponibilidade
- 11 Exercícios Resolvidos

Dependência do Software

A forte dependência de sistemas de software em praticamente todos os aspectos da vida profissional e pessoal implica que:

- Esperamos que o software esteja disponível quando necessário;
- Funcione no início da manhã, à noite, nos finais de semana e feriados;
- Opere continuamente (24h por dia, 7 dias por semana);
- Preserve dados e informações pessoais;
- Execute suas funções sem falhas ou interrupções.

Evolução da Confiabilidade

O avanço de:

- Técnicas de Engenharia de Software;
- Linguagens de programação mais robustas;
- Processos eficazes de garantia da qualidade;

resultou em melhorias significativas na confiabilidade nas últimas décadas.

Desafios Atuais

Apesar dos avanços:

- Falhas de sistema ainda ocorrem;
- A disponibilidade pode ser comprometida;
- Resultados incorretos podem ser produzidos.

1. Erro Humano (Engano)



Definição

Comportamento ou ação humana que resulta na introdução de defeitos em um sistema.

Exemplo nas Estações Meteorológicas:

- O programador estabelece a lógica: $T_{transmissão} = T_{atual} + 1h$.
- **A Falha de Raciocínio:** O desenvolvedor não considera a transição de ciclo do relógio de 24 horas.
- Ocorre no nível cognitivo do desenvolvedor.

2. Defeito de Sistema (Bug)

Definição

Uma característica física ou lógica no software que pode levar a um erro de sistema quando executada.

Representação em Java:

```
public class EstacaoMeteorologica {  
    private int horarioTransmissao;  
  
    public void agendarProxima(int horarioAtual) {  
        // O DEFEITO: Falta logica para resetar o dia (24h)  
        this.horarioTransmissao = horarioAtual + 1;  
  
        // Se horarioAtual for 23, o estado interno  
        // passara a ser 24 (Estado de Erro).  
    }  
}
```

- O defeito reside na **ausência** de uma condicional que trate o limite:
if (novoHorario >= 24) ...

3. Erro de Sistema (Estado Errôneo)

Definição

Um estado interno incorreto do sistema durante sua execução, resultante de um defeito.



Figura: Estado de erro no sistema

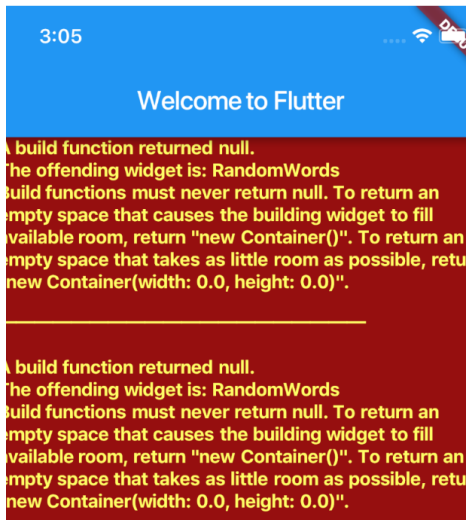
Manifestação em Tempo de Execução:

- O código defeituoso é executado quando o relógio marca 23h30.
- O valor da variável `horarioDeTransmissao` torna-se **24h30**.
- No sistema de 24h, o valor correto deveria ser **00h30**.
- O sistema ainda não "travou", mas seu estado interno está corrompido.

4. Falha de Sistema

Definição

Um evento em que o sistema não presta o serviço esperado pelo usuário final.



4. Falha de Sistema

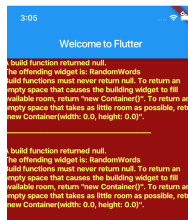


Figura: falha

O Impacto Final:

- O escalonador de tarefas busca por um horário válido para transmitir.
- Como **24h30** é um horário inexistente/inválido para o protocolo, a transmissão não ocorre.
- **Resultado:** Perda de dados e interrupção do serviço meteorológico.

Os defeitos de sistema não resultam necessariamente em erros, que por sua vez não resultam necessariamente em falhas de sistema

Defeitos $\nrightarrow$ Erros $\nrightarrow$ Falhas

Um sistema pode conter defeitos que nunca se manifestam como erros durante a execução. Da mesma forma, erros podem ocorrer sem causar falhas visíveis para os usuários.

1. O Defeito pode nunca ser executado

- Um **defeito** (bug no código) só se torna um **erro** (estado inválido) se a linha de código específica for processada.
- Dependendo do uso do software, certas funções ou ramos condicionais (*if/else*) podem nunca ser ativados.

Exemplo: Variável não inicializada

```
public void processarDados(boolean condicaoRara) {  
    int valor; // Defeito: Nao inicializada  
    if (condicaoRara) {  
        System.out.println(valor + 10); // Erro ocorre aqui  
    }  
}
```

Resultado: Se `condicaoRara` for sempre `false`, o defeito permanece latente e o sistema nunca entra em estado de erro.

2. Erros Transitórios

- Um **erro** (estado interno incorreto) pode existir temporariamente na memória, mas ser "atropelado" antes de causar uma **falha** visível.
- O estado é reiniciado por uma nova entrada ou processamento subsequente.

Cenário de Sobreposição

- ❶ O sistema calcula um horário de transmissão inválido: 24h15 (**Estado de Erro**).
- ❷ Antes do horário de transmissão chegar, o operador altera manualmente a configuração ou o sistema recebe um *reset* de sincronização.
- ❸ O valor é atualizado para 00h20 (**Estado Válido**).

Conclusão: O erro existiu, mas foi efêmero e não gerou impacto externo.

3. Mecanismos de Tolerância a Falhas

- Sistemas robustos são projetados para prever que erros ocorrerão.
- Mecanismos de **detecção** e **recuperação** impedem que o estado de erro se transforme em uma falha de serviço.

Estratégias Comuns

- **Validação de Dados:** O sistema checka se o valor é plausível antes de usá-lo.
- **Redundância:** Outro módulo calcula o mesmo dado e compara os resultados.
- **Exceções:** O erro é capturado (try-catch) e tratado graciosamente.

Corrigir defeitos em recursos não utilizados não afeta a confiabilidade do sistema

A distinção entre **defeitos**, **erros** e **falhas** leva a três abordagens complementares utilizadas para melhorar a confiabilidade de um sistema:

- Identificação e correção de defeitos críticos;
- Prevenção de erros durante a execução;
- Minimização do impacto de falhas no serviço prestado.

1. Prevenção de Defeitos

Objetivo

Utilizar abordagens de projeto e implementação que evitem a introdução de erros humanos no código, minimizando defeitos desde a origem.

Técnicas e Abordagens:

- **Linguagens Fortemente Tipadas:** Permitem que o compilador detecte inconsistências antes da execução (ex: Java, Rust).
- **Evitar Construtos Propensos a Erros:** Minimizar o uso de ponteiros ou manipulação direta de memória.
- **Padrões de Projeto:** Uso de boas práticas de arquitetura para reduzir a complexidade.

"Menos defeitos introduzidos significam menos chances de falhas operacionais."

Linguagem C

- Gestão Manual.
- Ponteiros Diretos.
- Verificação Fraca.

```
int arr[5];  
arr[10] = 100;  
// Defeito!  
// Acesso invalido.
```

Java

- Gestão Automática.
- Sem Ponteiros.
- Verificação Forte.

```
int[] arr = new int[5];  
arr[10] = 100;  
// Excecao!  
// Erro detectado.
```

Conclusão

O Java interrompe o erro antes que ele vire uma falha catastrófica, enquanto o C permite a execução em estado corrompido.

2. Detecção e Correção de Defeitos

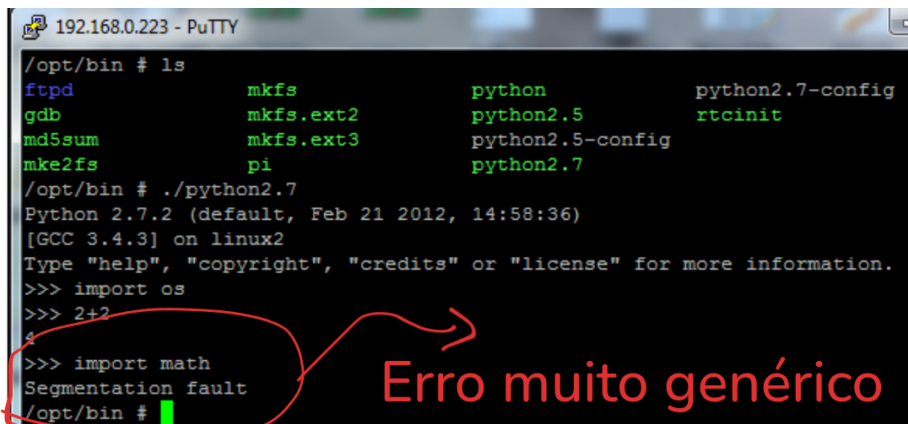
Objetivo

Descobrir e remover defeitos remanescentes através de Verificação e Validação (V&V) antes da implantação do sistema.

Estratégias Principais:

- **Análise Estática:** Revisão de código e ferramentas que buscam padrões de erro sem executar o programa.
- **Teste Sistemático:** Execução de casos de teste para garantir que o software atenda aos requisitos.
- **Depuração (Debugging):** Localização e correção da causa raiz dos defeitos encontrados.
- **Importância:** Fundamental para convencer *stakeholders* e órgãos reguladores sobre a dependabilidade do sistema.

1. Erro Humano (Engano)



A screenshot of a PuTTY terminal window titled "192.168.0.223 - PuTTY". The terminal shows a user at the "/opt/bin" prompt running "ls", which lists various files including "python", "python2.5", "python2.7", and their respective "-config" files. The user then runs "./python2.7", which starts the Python 2.7.2 interpreter. The user enters "import os", "2+2", and "import math". The last command, "import math", results in a "Segmentation fault" error. A red circle is drawn around the "import math" command and the error message, with a red arrow pointing from the circle to the text "Erro muito genérico" (Very generic error) written in red.

```
/opt/bin # ls
ftpd          mkfs          python        python2.7-config
gdb           mkfs.ext2    python2.5     rtcinit
md5sum        mkfs.ext3    python2.5-config
mke2fs        pi           python2.7
/opt/bin # ./python2.7
Python 2.7.2 (default, Feb 21 2012, 14:58:36)
[GCC 3.4.3] on linux2
Type "help", "copyright", "credits" or "license" for more information.
>>> import os
>>> 2+2
4
>>> import math
Segmentation fault
/opt/bin #
```

Erro muito genérico

3. Tolerância a Defeitos

Objetivo

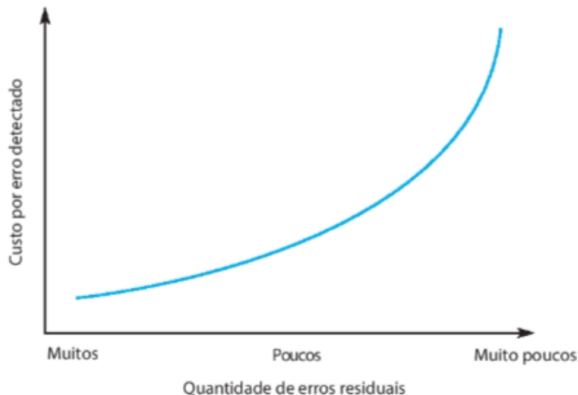
Garantir que, mesmo na presença de defeitos ou estados inesperados, o sistema consiga se recuperar e evitar uma falha total.

Níveis de Implementação:

- **Abordagens Simples:** Verificações em tempo de execução (*runtime checks*) e tratamento de exceções.
- **Sistemas Críticos:** Uso de redundância de hardware e software (Arquiteturas Tolerantes a Falhas).
- **Gerenciamento de Erros:** Detecção de estados inválidos e retorno a um estado seguro conhecido.

Essencial quando são necessários níveis altíssimos de disponibilidade e confiabilidade.

Custo de Detecção vs. Erros Residuais



Análise da Curva

- **Custo Exponencial:** Remover os últimos erros é drasticamente mais caro.
- **Decisão de Projeto:** Onde parar a detecção depende da criticidade do sistema.

Sistemas de altíssima disponibilidade operam na extremidade direita da curva.

O Dilema Econômico da Confiabilidade

- **Custo Exponencial:** O esforço para encontrar e remover defeitos remanescentes aumenta drasticamente conforme o sistema se torna mais confiável.
- **Ponto de Retorno Decrescente:** Gasta-se cada vez mais tempo e esforço para encontrar cada vez menos defeitos, tornando o processo injustificável em certo ponto.

Justificativa Econômica

As empresas aceitam **defeitos residuais** porque o custo da falha pode ser menor que o custo de remoção pré-entrega.

Níveis de Tolerância

- **Produtos de prateleira:** Nível de defeitos relativamente alto.
- **Sistemas críticos:** Nível de defeitos bem mais baixo.

Além da Economia

A decisão de lançar um software não é apenas financeira; deve-se considerar a **aceitabilidade social e política** de uma possível falha.



1. Confiabilidade (Reliability)

A probabilidade de operação **isenta de falhas** ao longo de um período especificado, em um determinado ambiente, para um propósito específico.

2. Disponibilidade (Availability)

A probabilidade de que um sistema, em algum momento, estará operacional e será capaz de prestar os serviços requisitados.

- Enquanto a **confiabilidade** foca na continuidade do serviço, a **disponibilidade** foca na prontidão do sistema quando solicitado.

A Confiabilidade não é um Valor Absoluto

A confiabilidade depende de **onde** e de **como** o sistema é utilizado.

Ambiente de Escritório

- Usuários seguem instruções.
- Uso restrito e previsível.
- **Resultado:** Menos falhas percebidas.

Ambiente Universitário

- Alunos exploram limites.
- Usos inesperados do sistema.
- **Resultado:** Falhas que não ocorreram no escritório.

"A percepção de confiabilidade muda drasticamente conforme o perfil de uso".

A Falha como Julgamento do Usuário

- A falha não é algo que possa ser definido de forma puramente objetiva; trata-se de um **juízo** feito por quem utiliza o sistema.

1 Especificações Incompletas ou Incorretas:

- Engenheiros interpretam o comportamento sem serem especialistas no domínio.
- O software pode seguir a especificação à risca, mas ainda ser considerado "defeituoso" pelo usuário por não atender à sua expectativa real.

2 Lacuna de Comunicação:

- Raramente alguém além dos desenvolvedores lê os documentos de especificação.
- Usuários preveem comportamentos baseados na intuição, enquanto a especificação pode determinar algo oposto.

Conclusão

Como as percepções variam, os usuários não possuem a mesma impressão sobre a confiabilidade de um sistema.

Sistema como Mapeamento de Entradas/Saídas



Exemplo Prático

Dada a entrada de uma URL, um navegador web produz a exibição da página solicitada como saída.

- A maioria das entradas processadas não resulta em falhas.
- Entretanto, certas combinações de entradas (E_e) provocam falhas ou saídas errôneas.

O Impacto da Utilização na Confiabilidade

- A confiabilidade depende de quantas entradas do conjunto E_e são executadas no dia a dia.
- O conjunto E_e representa as entradas que provocam a execução de código defeituoso e erros de sistema.

Falhas Frequentes

Ocorrem se as entradas de E_e fazem parte das funcionalidades mais utilizadas do sistema.

Falhas Raras

Ocorrem se as entradas de E_e pertencem a códigos raramente acessados pelos usuários.

Percepção de Confiabilidade por Usuário

- Defeitos que afetam um usuário podem nunca aparecer no modo de trabalho de outro.

Usuário 2: Experimenta falhas, pois seu conjunto de entradas tem interseção com o conjunto errôneo E_e .

Usuários 1 e 3: Nunca utilizam as entradas do conjunto errôneo.



Conclusão

Para os usuários 1 e 3, o software sempre parecerá 100% confiável, mesmo que contenha defeitos.

- A disponibilidade não depende apenas do número de falhas, mas também do **tempo necessário para consertar** os defeitos.

Métrica	Sistema A	Sistema B
Frequência de Falha	1 por ano	1 por mês
Tempo de Reinício	6 horas (360 min)	5 minutos
Parada Total/Ano	360 minutos	60 minutos

Conclusão do Exemplo

Embora o Sistema A pareça mais confiável (falha menos), o **Sistema B tem uma disponibilidade muito maior** ao longo do ano.

- A métrica de porcentagem simples de tempo disponível não reflete a **perturbação real** causada ao usuário.
- O **momento** em que a falha ocorre é um fator determinante.

Cenário A: Baixo Impacto

Indisponibilidade de **1 hora** diária entre 3h00 e 4h00 da manhã. Afeta poucos usuários.

Cenário B: Alto Impacto

Indisponibilidade de apenas **10 minutos** durante o horário comercial. Efeito muito maior nos usuários.

- Confiabilidade e disponibilidade estão relacionadas, mas sua importância relativa varia conforme o uso.

Alta Disponibilidade: Requisito essencial quando os usuários esperam por um **serviço contínuo**. O sistema deve estar pronto sempre que houver demanda.

Recuperação Rápida: Se o sistema se recupera de falhas sem perda de dados, o impacto percebido pelo usuário pode ser minimizado, mesmo com falhas frequentes.

"Uma falha rápida e segura pode ser preferível a uma falha rara, mas catastrófica."

Definição

É a propriedade de um sistema de computação que permite confiar justificadamente no serviço que ele entrega.

Os Pilares da Dependabilidade:

- **Disponibilidade:** Prontidão para o uso.
- **Segurança (Safety):** Ausência de consequências catastróficas.
- **Confiabilidade:** Continuidade de serviço isento de falhas.
- **Proteção (Security):** Resistência a intrusões e danos.
- É uma **propriedade holística**: depende não apenas do software, mas também do hardware e das pessoas (operadores).
- Sistemas de alta dependabilidade são projetados para sobreviver a erros sem causar falhas.

- A dependabilidade exige atenção minuciosa na **derivação** e no **ajuste** dos requisitos do sistema.

1. Requisitos Funcionais

Definem recursos de:

- Verificação e recuperação.
- Proteção contra falhas e ataques externos.
- **Visão Holística:** A confiabilidade global depende do hardware, software e dos operadores.
- **Compensação de Erros:** O software deve incluir requisitos que ajudem na detecção e recuperação de falhas de hardware e erros humanos.

2. Requisitos Não Funcionais

Definem as metas de:

- **Confiabilidade** necessária.
- **Disponibilidade** necessária.

1 POFOD (Probabilidade de Falha Sob Demanda)

- Probabilidade de uma requisição de serviço resultar em falha.
- **Exemplo:** $POFOD = 0,001$ indica 1 chance em 1.000 de falha ao acionar o sistema.

2 ROCOF (Taxa de Ocorrência de Falhas)

- Número provável de falhas em um período de tempo ou execuções.
- **MTTF (Mean Time To Failure):** É o inverso da ROCOF ($1/ROCOF$).
- **Exemplo:** Se $ROCOF = 2$ falhas/hora, então $MTTF = 30$ minutos.

Diferença Crucial

A POFOD é ideal para sistemas de proteção (ex: freio ABS), enquanto a ROCOF/MTTF é melhor para sistemas de uso contínuo (ex: servidores).

MTTR - Mean Time To Repair

Representa a média do tempo gasto para realizar o reparo de falhas:

$$MTTR = \frac{T_{total_manutencao}}{N_{reparos}}$$

Quanto menor o valor, maior a **manutenibilidade** do sistema.

Disponibilidade (AVAIL)

- Probabilidade de o sistema estar operacional no momento da demanda.
- Medida geralmente como uma percentagem do tempo total de operação.

Formula

$$AVAIL = \frac{MTTF}{MTTF + MTTR}$$

Tabela: Impacto dos níveis de Disponibilidade em 24h

AVAIL	Porcentagem	Tempo de Indisponibilidade
0,90	90%	144 minutos (2h 24min)
0,99	99%	14,4 minutos
0,999	99,9%	84 segundos
0,9999	99,99%	8,4 segundos

- Os **"Nove"**: Na indústria, fala-se em "quatro nove"(99,99%) para indicar sistemas de altíssima disponibilidade.

Enunciado

Um sistema de acionamento de emergência de um gerador foi testado 500 vezes. Em 2 dessas ocasiões, o gerador não ligou quando solicitado. Qual é a **POFOD** deste sistema?

Exercício 1: POFOD

Enunciado

Um sistema de acionamento de emergência de um gerador foi testado 500 vezes. Em 2 dessas ocasiões, o gerador não ligou quando solicitado. Qual é a **POFOD** deste sistema?

Resolução

$$POFOD = \frac{\text{Número de Falhas}}{\text{Número de Demandas}}$$

$$POFOD = \frac{2}{500} = 0,004$$

Resposta: A probabilidade de falha sob demanda é de **0,4%** (ou 4 falhas a cada 1.000 demandas).

Enunciado

Um servidor web registrou 12 falhas durante um período de operação de 48 horas. Calcule a taxa de ocorrência de falhas (**ROCOF**) e o tempo médio até a falha (**MTTF**).

Exercício 2: ROCOF e MTTF

Enunciado

Um servidor web registrou 12 falhas durante um período de operação de 48 horas. Calcule a taxa de ocorrência de falhas (**ROCOF**) e o tempo médio até a falha (**MTTF**).

Resolução

$$ROCOF = \frac{\text{Falhas}}{\text{Tempo}} = \frac{12}{48} = 0,25 \text{ falhas/hora}$$

$$MTTF = \frac{1}{ROCOF} = \frac{1}{0,25} = 4 \text{ horas}$$

Resposta: O sistema falha, em média, a cada **4 horas**.

Exercício 3: Disponibilidade Percetual

Enunciado

Um sistema de banco de dados deve operar 24h por dia. Na última semana (7 dias), ele ficou fora do ar por um total de 70 minutos para manutenção emergencial. Qual foi a sua disponibilidade (**AVAIL**)?

Exercício 3: Disponibilidade Percetual

Enunciado

Um sistema de banco de dados deve operar 24h por dia. Na última semana (7 dias), ele ficou fora do ar por um total de 70 minutos para manutenção emergencial. Qual foi a sua disponibilidade (**AVAIL**)?

Resolução

Tempo total (min): $7 \times 24 \times 60 = 10.080$ min

Tempo operacional: $10.080 - 70 = 10.010$ min

$$AVAIL = \frac{10.010}{10.080} \approx 0,9930$$

Resposta: A disponibilidade foi de aproximadamente **99,30%**.

Exercício 3: Disponibilidade Percetual

Enunciado

Um sistema de banco de dados deve operar 24h por dia. Na última semana (7 dias), ele ficou fora do ar por um total de 70 minutos para manutenção emergencial. Qual foi a sua disponibilidade (**AVAIL**)?

Exercício 3: Disponibilidade Percetual

Enunciado

Um sistema de banco de dados deve operar 24h por dia. Na última semana (7 dias), ele ficou fora do ar por um total de 70 minutos para manutenção emergencial. Qual foi a sua disponibilidade (**AVAIL**)?

Resolução

Tempo total (min): $7 \times 24 \times 60 = 10.080$ min

Tempo operacional: $10.080 - 70 = 10.010$ min

$$AVAIL = \frac{10.010}{10.080} \approx 0,9930$$

Resposta: A disponibilidade foi de aproximadamente **99,30%**.

Exercício 4: O Desafio dos "Três Noves"

Enunciado

Se um cliente exige um contrato de nível de serviço (SLA) com disponibilidade de **0,999**, quanto tempo de parada (Downtime) é permitido em um mês comercial de 30 dias?

Exercício 4: O Desafio dos "Três Noves"

Enunciado

Se um cliente exige um contrato de nível de serviço (SLA) com disponibilidade de **0,999**, quanto tempo de parada (Downtime) é permitido em um mês comercial de 30 dias?

Resolução

Tempo total no mês: $30 \times 1.440 \text{ min} = 43.200 \text{ min}$

Indisponibilidade permitida: $1 - 0,999 = 0,001$

Downtime = $43.200 \times 0,001 = 43,2 \text{ minutos}$

Resposta: O sistema pode ficar fora do ar por no máximo **43,2 minutos** por mês.

Exercício 5: Confiabilidade vs. Disponibilidade

Enunciado

O Sistema X tem $MTTF = 100h$ e leva $10h$ para reiniciar. O Sistema Y tem $MTTF = 20h$ e leva $1h$ para reiniciar. Qual deles tem a maior disponibilidade?

Exercício 5: Confiabilidade vs. Disponibilidade

Enunciado

O Sistema X tem $MTTF = 100h$ e leva $10h$ para reiniciar. O Sistema Y tem $MTTF = 20h$ e leva $1h$ para reiniciar. Qual deles tem a maior disponibilidade?

Resolução

$$AVAIL = \frac{MTTF}{MTTF + MTTR}$$

$$AVAIL(X) = \frac{100}{100+10} = \frac{100}{110} \approx 0,909$$

$$AVAIL(Y) = \frac{20}{20+1} = \frac{20}{21} \approx 0,952$$

Resposta: O **Sistema Y** tem maior disponibilidade, apesar de falhar 5 vezes mais que o Sistema X.